

PYTHON

Session 6: Introduction to Python and Functions

1. What is the correct file extension for Python files?

- A. .py
- B. .python
- C. .pyth
- D. .pt

Answer: A

2. Which of the following is a valid variable name in Python?

- A. 2variable
- B. variable@name
- C. variable_name
- D. variable-name

Answer: C

3. What does the `len()` function in Python return?

- A. The length of a variable name
- B. The length of a string or collection
- C. The total number of functions in a program
- D. None of the above

Answer: B

4. How do you define a function in Python?

- A. `function myFunction():`
- B. `def myFunction():`
- C. `define myFunction():`
- D. `myFunction() = def:`

Answer: B

5. What keyword is used to return a value from a function?

- A. return
- B. output

- C. result
 - D. yield
- Answer: A**
-

6. Which of these is not a valid way to call a Python function?

- A. myFunction()
- B. myFunction(arguments)
- C. myFunction[]
- D. myFunction()

Answer: C

7. What will be the output of the following code?

```
def add(a, b):  
    return a + b  
  
print(add(5, 3))
```

- A. 8
- B. 53
- C. Error
- D. None

Answer: A

8. Which of the following is not a built-in function in Python?

- A. print()
- B. input()
- C. main()
- D. type()

Answer: C

9. What is the output of the following code?

```
def greet():  
    print("Hello, World!")  
  
result = greet()  
print(result)
```

- A. Hello, World!
- B. Hello, World! None

- C. None
 - D. Error
- Answer: B**
-

10. Which statement is true about default parameters in Python functions?

- A. Default parameters must always come before non-default parameters.
- B. Default parameters are optional when calling the function.
- C. A function cannot have default parameters.
- D. Default parameters are mandatory.

Answer: B

11. How is an anonymous function created in Python?

- A. Using the `def` keyword
- B. Using the `lambda` keyword
- C. Using the `function` keyword
- D. Using the `anonymous` keyword

Answer: B

12. What will the following code print?

```
x = 5

def example():
    global x
    x = 10

example()
print(x)
```

- A. 5
- B. 10
- C. Error
- D. None

Answer: B

13. What is the output of this code?

```
def multiply(a, b=2):
    return a * b

print(multiply(3))
```

- A. 6
- B. 3
- C. Error
- D. None

Answer: A

14. Which of the following is not true about Python functions?

- A. Functions are reusable.
- B. A function can return multiple values.
- C. Functions cannot be nested.
- D. Functions can be assigned to variables.

Answer: C

15. What will the output be?

```
def test(a, b):  
    return a * b, a + b  
  
result = test(2, 3)  
print(result)
```

- A. 6, 5
- B. (6, 5)
- C. Error
- D. None

Answer: B

16. Which of the following is the correct syntax to pass an unknown number of positional arguments to a function?

- A. `def func(*args):`
- B. `def func(**args):`
- C. `def func(args*):`
- D. `def func(args**):`

Answer: A

17. What will this code output?

```
def outer():  
    def inner():  
        return "Inner Function"  
    return inner
```

```
result = outer()
print(result())
```

- A. Inner Function
 - B. `<function outer.<locals>.inner>`
 - C. None
 - D. Error
- Answer: A**
-

18. What is a docstring in Python?

- A. A comment at the start of a file
 - B. A string that documents a function, class, or module
 - C. A variable that holds the name of the function
 - D. None of the above
- Answer: B**
-

19. What does the `*args` syntax do?

- A. Accepts keyword arguments
 - B. Allows passing a variable number of positional arguments
 - C. Declares a global variable
 - D. Handles exception arguments
- Answer: B**
-

20. What is the return type of the `type()` function in Python?

- A. `str`
 - B. `int`
 - C. `type`
 - D. None
- Answer: C**
-

21. What happens when a function does not include a `return` statement?

- A. It returns `None`.
 - B. It raises an error.
 - C. It returns an empty string.
 - D. It executes indefinitely.
- Answer: A**

22. How can you specify default arguments in a function?

- A. By adding the keyword `default`
- B. By assigning values in the function definition
- C. By calling the function with `None`
- D. By using `args`

Answer: B

23. What is the output of this code?

```
def demo(a, b=5, c=10):  
    return a + b + c  
  
print(demo(1, c=20))
```

- A. 26
- B. 36
- C. 16
- D. Error

Answer: A

24. Which of the following statements is true about Python's `return` keyword?

- A. A function must have a `return` statement.
- B. A function can return only one value.
- C. A function can have multiple `return` statements.
- D. A function cannot return `None`.

Answer: C

25. What is the main purpose of functions in Python?

- A. Reduce code complexity and reuse code
- B. Only for debugging
- C. Enhance program runtime performance
- D. None of the above

Answer: A

Session 7 & 8: Lists and Dictionaries, with answers:

1. What will be the output of the following code?

```
list1 = [1, 2, 3]
list2 = list1
list1[0] = 5
print(list2)
```

- A. [1, 2, 3]
- B. [5, 2, 3]
- C. Error
- D. None

Answer: B

2. Which of the following methods is used to add an item to a list?

- A. add()
- B. insert()
- C. append()
- D. push()

Answer: C

3. What does the `list.extend()` method do?

- A. Adds a new item at the end of the list
- B. Adds all elements of another iterable to the list
- C. Removes the last element of the list
- D. Replaces an item at a specific index

Answer: B

4. Which of these will create an empty dictionary?

- A. dict = {}
- B. dict = []
- C. dict = set()
- D. dict = ()

Answer: A

5. What will the following code output?

```
my_dict = {"a": 1, "b": 2}
print(my_dict.get("c"))
```

- A. 0
- B. None
- C. KeyError
- D. c

Answer: B

6. What happens when two keys in a dictionary are the same?

- A. An error is raised.
- B. The last key-value pair is retained.
- C. Both key-value pairs are stored.
- D. The first key-value pair is retained.

Answer: B

7. What will the following code output?

```
a = [1, 2, 3, 4, 5]
print(a[::2])
```

- A. [1, 3, 5]
- B. [1, 2, 3]
- C. [2, 4]
- D. Error

Answer: A

8. Which method removes an item by its index in a list?

- A. remove()
- B. pop()
- C. del
- D. discard()

Answer: B

9. What does the following code output?

```
my_list = [1, 2, 3]
my_list *= 2
print(my_list)
```

- A. [1, 2, 3, 1, 2, 3]
- B. [1, 2, 3]
- C. Error

D. [6, 4, 2]

Answer: A

10. Which of these methods can be used to iterate over both keys and values in a dictionary?

A. items()

B. keys()

C. values()

D. get()

Answer: A

11. What is the output of the following code?

```
my_dict = {1: 'a', 2: 'b', 3: 'c'}  
print(2 in my_dict)
```

A. True

B. False

C. Error

D. None

Answer: A

12. Which of the following is not a valid dictionary method?

A. update()

B. clear()

C. sort()

D. popitem()

Answer: C

13. How can you create a list with 10 elements, all initialized to 0?

A. [0, 10]

B. [0]*10

C. list(0, 10)

D. list(0)*10

Answer: B

14. What will the following code output?

```
my_list = [10, 20, 30]
my_list.insert(1, 15)
print(my_list)
```

- A. [10, 15, 20, 30]
- B. [10, 20, 30, 15]
- C. [15, 10, 20, 30]
- D. Error

Answer: A

15. How can you create a dictionary from two lists: `keys` and `values`?

- A. `dict(keys, values)`
- B. `keys:values`
- C. `dict(zip(keys, values))`
- D. `dict(keys=values)`

Answer: C

16. What is the result of the following code?

```
a = {"x": 10, "y": 20}
b = {"z": 30}
a.update(b)
print(a)
```

- A. {"x": 10, "y": 20}
- B. {"z": 30}
- C. {"x": 10, "y": 20, "z": 30}
- D. Error

Answer: C

17. Which method is used to remove an item by key from a dictionary?

- A. `del`
- B. `remove()`
- C. `discard()`
- D. `pop()`

Answer: D

18. What is the output of this code?

```
my_dict = {"a": 1, "b": 2}
del my_dict["a"]
print(my_dict)
```

- A. {"b": 2}
- B. {"a": 1, "b": 2}
- C. Error
- D. None

Answer: A

19. What will the following code output?

```
my_list = [10, 20, 30]
print(my_list.index(20))
```

- A. 2
- B. 1
- C. 20
- D. Error

Answer: B

20. How can you merge two dictionaries in Python 3.9+?

- A. dict1 + dict2
- B. dict1 | dict2
- C. dict1.merge(dict2)
- D. merge(dict1, dict2)

Answer: B

21. What is the output of the following code?

```
a = [1, 2, 3]
b = a.copy()
b.append(4)
print(a, b)
```

- A. [1, 2, 3] [1, 2, 3]
- B. [1, 2, 3, 4] [1, 2, 3, 4]
- C. [1, 2, 3] [1, 2, 3, 4]
- D. Error

Answer: C

22. What is the output of this code?

```
my_list = ["a", "b", "c", "d"]  
print(my_list[::-1])
```

- A. ["a", "b", "c", "d"]
- B. ["d", "c", "b", "a"]
- C. Error
- D. None

Answer: B

23. How do you check if a key exists in a dictionary?

- A. `if key in dict`
- B. `if key in dict.keys()`
- C. Both A and B
- D. None of the above

Answer: C

24. Which of the following is a mutable data type?

- A. `list`
- B. `tuple`
- C. `str`
- D. `int`

Answer: A

25. What does the `sorted()` function return when used on a dictionary?

- A. A list of keys in ascending order
- B. A sorted dictionary
- C. A list of values in ascending order
- D. An error

Answer: A

Session 9: Tuples and String Manipulation, with answers:

1. What is the output of the following code?

```
my_tuple = (1, 2, 3, 4)  
print(my_tuple[1:3])
```

- A. (1, 2)
- B. (2, 3)
- C. (1, 3)
- D. (2, 3, 4)

Answer: B

2. Which of the following operations is not allowed on a tuple?

- A. Indexing
- B. Slicing
- C. Changing an element
- D. Iterating

Answer: C

3. What will the following code output?

```
my_tuple = (1, 2, 3, 4)
my_tuple[0] = 5
```

- A. Error
- B. (5, 2, 3, 4)
- C. (1, 2, 3, 4)
- D. None

Answer: A

4. How can you create a tuple with only one element?

- A. (1)
- B. 1,
- C. (1,)
- D. []

Answer: C

5. Which of the following is the correct syntax to concatenate two tuples?

- A. tuple1 + tuple2
- B. tuple1.append(tuple2)
- C. tuple1.concat(tuple2)
- D. tuple1.extend(tuple2)

Answer: A

6. What does the `len()` function return when applied to a tuple?

- A. The sum of elements in the tuple
- B. The first element of the tuple
- C. The number of elements in the tuple
- D. The last element of the tuple

Answer: C

7. Which of the following methods is not applicable to tuples?

- A. `count()`
- B. `index()`
- C. `sort()`
- D. `len()`

Answer: C

8. What will the following code output?

```
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple[::-1])
```

- A. (1, 2, 3, 4, 5)
- B. (5, 4, 3, 2, 1)
- C. Error
- D. (5, 4, 3)

Answer: B

9. Which of the following functions can be used to join strings in Python?

- A. `join()`
- B. `split()`
- C. `concat()`
- D. `combine()`

Answer: A

10. What is the output of the following code?

```
str1 = "Python"
str2 = "Programming"
print(str1 + str2)
```

- A. Python Programming
- B. PythonProgramming
- C. Error
- D. None

Answer: B

11. What does the `split()` method do in Python?

- A. Joins multiple strings into one
- B. Divides a string into a list of substrings
- C. Removes whitespace from a string
- D. Concatenates a string

Answer: B

12. What is the output of this code?

```
string = "Hello"  
print(string[1:4])
```

- A. Hell
- B. ell
- C. lo
- D. Hello

Answer: B

13. Which of the following methods is used to find the index of a substring in a string?

- A. `find()`
- B. `index()`
- C. `search()`
- D. `locate()`

Answer: B

14. What will be the output of the following code?

```
s = "apple"  
print(s.upper())
```

- A. apple
- B. APPLE
- C. Error

D. None

Answer: B

15. What is the correct way to convert a string to a list in Python?

A. `list(str)`

B. `str.split()`

C. `str.toList()`

D. `listOf(str)`

Answer: A

16. What is the result of the expression `'Hello' * 3`?

A. HelloHelloHello

B. Hello 3 times

C. Hello

D. Error

Answer: A

17. What does the `join()` method do in Python?

A. Joins two strings

B. Joins a list of strings into a single string

C. Splits a string into a list

D. Joins a string with a separator

Answer: B

18. What will the following code output?

```
str = "hello"  
print(str[-1])
```

A. h

B. o

C. Error

D. l

Answer: B

19. What is the result of the expression `tuple([1, 2, 3])`?

- A. [1, 2, 3]
- B. (1, 2, 3)
- C. (1, 2)
- D. Error

Answer: B

20. How would you check if a string contains only digits?

- A. `str.isnumeric()`
- B. `str.isdigit()`
- C. `str.isdecimal()`
- D. All of the above

Answer: D

21. What is the output of the following code?

```
my_string = "Python"  
print(my_string[0:4:2])
```

- A. Pto
- B. Pyn
- C. Pyt
- D. Ptn

Answer: A

22. How do you check if a tuple is empty in Python?

- A. `if len(tuple) == 0:`
- B. `if tuple == []:`
- C. `if not tuple:`
- D. All of the above

Answer: D

23. What is the result of the expression `tuple('abc')`?

- A. ('a', 'b', 'c')
- B. ['a', 'b', 'c']
- C. ('abc')
- D. Error

Answer: A

24. What is the purpose of the `count()` method in a tuple?

- A. Count the number of occurrences of a specified item
- B. Return the index of the specified item
- C. Count the total number of elements in the tuple
- D. Count the total number of tuples

Answer: A

25. What is the output of the following code?

```
str = "Good Morning"  
print(str[5:])
```

- A. Morning
- B. Good
- C. Good M
- D. Error

Answer: A

Session 10: Advanced Python Concepts, with answers:

1. What will the following code output?

```
def test(a, b=10):  
    return a + b  
  
print(test(5))
```

- A. None
- B. 5
- C. 15
- D. Error

Answer: C

2. What is the output of the following code?

```
def foo(*args):  
    return len(args)  
  
print(foo(1, 2, 3))
```

- A. 1
- B. 3

- C. None
 - D. Error
- Answer: B**
-

3. What does the `@staticmethod` decorator do?

- A. Converts a method into a class method
- B. Makes a method accessible without an instance
- C. Prevents method overriding
- D. Allows access to the instance within the method

Answer: B

4. What is the result of the following code?

```
class Example:
    def __init__(self):
        self.x = 5
    def __str__(self):
        return str(self.x)
```

```
obj = Example()
print(obj)
```

- A. 5
- B. <Example object at 0x123>
- C. None
- D. Error

Answer: A

5. Which of the following is used to create a generator in Python?

- A. `yield`
- B. `return`
- C. `def`
- D. `lambda`

Answer: A

6. What is the output of the following code?

```
def recursive(n):
    if n <= 0:
        return
    else:
```

```
        print(n)
        recursive(n-1)

recursive(3)
```

- A. 3 2 1
 - B. 1 2 3
 - C. None
 - D. Error
- Answer: A**
-

7. What does the `super()` function do in Python?

- A. Returns the parent class object
 - B. Returns the current class object
 - C. Used to access methods from other classes
 - D. Both A and C
- Answer: D**
-

8. Which of the following is not a valid Python iterator?

- A. `list`
 - B. `tuple`
 - C. `range`
 - D. `dict`
- Answer: D**
-

9. What does the `zip()` function do in Python?

- A. Combines multiple iterables element-wise
 - B. Unzips a zipped iterable
 - C. Compresses multiple strings into a single string
 - D. None of the above
- Answer: A**
-

10. What is the output of the following code?

```
def test(a, b):
    return a + b

result = test(10, 5)
print(result)
```

- A. 105
 - B. 15
 - C. None
 - D. Error
- Answer: B**
-

11. What will be the output of the following code?

```
def function():  
    try:  
        return 1  
    finally:  
        return 2  
  
print(function())
```

- A. 1
 - B. 2
 - C. None
 - D. Error
- Answer: B**
-

12. What is the output of this code?

```
x = [1, 2, 3]  
y = x  
y[0] = 10  
print(x)
```

- A. [10, 2, 3]
 - B. [1, 2, 3]
 - C. [10, 2, 3, 4]
 - D. Error
- Answer: A**
-

13. Which of the following statements is true about Python's list comprehension?

- A. It is slower than using a `for` loop
 - B. It is more concise and can result in cleaner code
 - C. It does not support filtering elements
 - D. It can only be used for integer data types
- Answer: B**
-

14. What is the output of the following code?

```
def test(x, y=2, z=3):  
    return x + y + z  
  
print(test(1, z=5))
```

- A. 6
- B. 8
- C. 10
- D. None

Answer: B

15. What does the `lambda` function in Python do?

- A. Returns a reference to a function
- B. Defines an anonymous function
- C. Raises an exception
- D. Allows defining a function with multiple statements

Answer: B

16. Which of the following is used to define a class in Python?

- A. `class`
- B. `object`
- C. `function`
- D. `def`

Answer: A

17. What is the output of this code?

```
x = {'a': 1, 'b': 2}  
y = {'b': 3, 'c': 4}  
z = x.update(y)  
print(z)
```

- A. {'a': 1, 'b': 3, 'c': 4}
- B. None
- C. Error
- D. {'a': 1}

Answer: B

18. What will the following code output?

```
def func(a, b, c):  
    return a * b + c  
  
result = func(2, 3, 4)  
print(result)
```

- A. 10
- B. 12
- C. 14
- D. Error

Answer: B

19. What is the correct syntax for defining a decorator in Python?

- A. `def decorator(func):`
- B. `def func() @decorator:`
- C. `@decorator def func():`
- D. `decorator def func():`

Answer: C

20. What is the output of the following code?

```
def generator():  
    yield 1  
    yield 2  
    yield 3  
  
gen = generator()  
for i in gen:  
    print(i)
```

- A. 1 2 3
- B. Error
- C. None
- D. 1

Answer: A

21. Which of the following is a Python data structure that stores unique elements?

- A. list
- B. set
- C. tuple

D. dict

Answer: B

22. How can you create a private variable in Python?

- A. Prefix the variable name with an underscore (`_`)
- B. Prefix the variable name with double underscores (`__`)
- C. Use the `private` keyword
- D. Python does not support private variables

Answer: B

23. What does the `finally` block do in exception handling?

- A. Executes when an exception is raised
- B. Executes if no exception is raised
- C. Executes regardless of whether an exception is raised or not
- D. Both A and B

Answer: C

24. Which of the following is the correct way to define a class method?

- A. `def method(cls):`
- B. `def method(self):`
- C. `@classmethod def method(self):`
- D. `@classmethod def method(cls):`

Answer: D

25. What is the output of the following code?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

- A. `[1, 2, 3, 4]`
- B. `[1, 2, 3]`
- C. `[4]`
- D. Error

Answer: A

Session 11: Regular Expressions, Scripting, and Functional Programming, with answers:

1. What is the output of the following regular expression code?

```
import re
result = re.findall(r'\d+', '123 abc 456')
print(result)
```

- A. ['123', '456']
- B. ['123', ' abc', '456']
- C. ['123 abc 456']
- D. []

Answer: A

2. Which of the following characters is used for matching any character in a regular expression?

- A. *
- B. +
- C. .
- D. ?

Answer: C

3. What does the `re.search()` function return?

- A. The first match of the pattern
- B. A match object for the pattern
- C. A list of all matches
- D. None if no match is found

Answer: B

4. Which regular expression pattern would match a string that starts with "hello" and ends with "world"?

- A. ^hello\$
- B. ^hello.*world\$
- C. hello.*world
- D. hello.world

Answer: B

5. What will the following code output?

```
import re
text = 'The rain in Spain'
match = re.findall(r'in', text)
print(match)
```

- A. ['in', 'in']
- B. ['in']
- C. ['rain']
- D. []

Answer: A

6. Which of the following will match an email address in Python using regular expressions?

- A. `r'\w+@\w+\.\w+'`
- B. `r'[a-z]+@[a-z]+\.[a-z]+'`
- C. `r'\S+@\S+\.\S+'`
- D. All of the above

Answer: D

7. What does the `re.sub()` function do in Python?

- A. Finds a pattern in the string
- B. Replaces a pattern in the string
- C. Splits a string by a pattern
- D. Returns a boolean indicating a match

Answer: B

8. Which Python keyword is used to define a lambda function?

- A. `def`
- B. `lambda`
- C. `function`
- D. `fn`

Answer: B

9. Which of the following is the correct syntax for using the `map()` function?

- A. `map(function, iterable)`
- B. `map(iterable, function)`
- C. `function.map(iterable)`

D. `map(function(iterable))`

Answer: A

10. What is the purpose of the `filter()` function in Python?

- A. Filters out specific elements from a list
- B. Returns all elements that satisfy a condition
- C. Removes non-matching elements from an iterable
- D. Both B and C

Answer: D

11. Which function would you use to return an iterator that aggregates elements from an iterable in Python?

- A. `map()`
- B. `filter()`
- C. `reduce()`
- D. `zip()`

Answer: C

12. What does the `re.match()` function do in Python?

- A. Returns a match object only if the pattern matches the beginning of the string
- B. Returns the first match anywhere in the string
- C. Finds all matches of the pattern
- D. Returns a list of all matches

Answer: A

13. Which regular expression pattern matches any whitespace character (spaces, tabs, newlines)?

- A. `\w`
- B. `\s`
- C. `\d`
- D. `\b`

Answer: B

14. Which of the following statements about Python scripting is true?

- A. Python scripts are compiled before execution
- B. Python scripts are interpreted line by line
- C. Python scripts cannot interact with external systems
- D. Python scripts require a complex build process

Answer: B

15. What is the output of the following code?

```
def square(x):  
    return x * x  
  
numbers = [1, 2, 3, 4]  
result = list(map(square, numbers))  
print(result)
```

- A. [1, 4, 9, 16]
- B. [1, 2, 3, 4]
- C. [2, 4, 6, 8]
- D. Error

Answer: A

16. What is the output of the following code?

```
def is_even(n):  
    return n % 2 == 0  
  
numbers = [1, 2, 3, 4, 5, 6]  
result = list(filter(is_even, numbers))  
print(result)
```

- A. [1, 3, 5]
- B. [2, 4, 6]
- C. [2, 3, 4, 5, 6]
- D. []

Answer: B

17. What is the result of the following code?

```
def add(x, y):  
    return x + y  
  
result = add(2, 3)  
print(result)
```

- A. None
- B. 5

C. 23

D. Error

Answer: B

18. Which of the following functional programming concepts is not supported directly in Python?

A. Higher-order functions

B. First-class functions

C. Lazy evaluation

D. Pure functions

Answer: C

19. What does the `re.findall()` function return?

A. A list of all matches in the string

B. A boolean value indicating whether a match was found

C. The first match found in the string

D. A tuple of all matches

Answer: A

20. What will the following code output?

```
import re
text = 'Python123 Regex456'
match = re.sub(r'\d+', '', text)
print(match)
```

A. Python Regex

B. Python123 Regex456

C. Python Regex

D. Error

Answer: A

21. What is the correct syntax for a regular expression that matches a string with exactly 3 digits?

A. `\d{3}`

B. `\d{3,}`

C. `\d*{3}`

D. `\d[3]`

Answer: A

22. Which of the following statements about list comprehensions is true?

- A. They are always faster than using a `for` loop
- B. They can only be used for lists
- C. They cannot include conditions
- D. They allow concise and readable expressions

Answer: D

23. What is the result of this Python code?

```
def test(*args):  
    return sum(args)  
  
result = test(1, 2, 3)  
print(result)
```

- A. 1
- B. 6
- C. None
- D. Error

Answer: B

24. Which of the following statements is correct about the use of the `re` module in Python?

- A. The `re` module only works with strings
- B. The `re` module is used to perform operations on files
- C. The `re` module is used to manipulate data structures
- D. The `re` module cannot work with regular expressions in Python

Answer: A

25. What will the following code output?

```
def multiply(x, y=10):  
    return x * y  
  
print(multiply(2))
```

- A. 10
- B. 20
- C. 2

D. Error

Answer: B

Session 12: Object-Oriented Programming Concepts, with answers:

1. What is the output of the following code?

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        print("Woof")

d = Dog()
d.sound()
```

A. Animal sound

B. Woof

C. Error

D. None

Answer: B

2. Which of the following is the correct way to create an object of a class?

A. object = class()

B. object = new class()

C. object = create(class)

D. object = class.create()

Answer: A

3. What is the purpose of the `self` keyword in Python?

A. To refer to the current object of the class

B. To refer to a global variable

C. To define a method

D. To create an instance of a class

Answer: A

4. What is the output of the following code?

```
class Person:
    def __init__(self, name):
        self.name = name

p = Person("Alice")
print(p.name)
```

- A. None
 - B. Alice
 - C. person.name
 - D. Error
- Answer: B**
-

5. Which of the following is a key feature of Object-Oriented Programming?

- A. Abstraction
 - B. Encapsulation
 - C. Inheritance
 - D. All of the above
- Answer: D**
-

6. Which method is called when an object is created in a Python class?

- A. `__init__()`
 - B. `__new__()`
 - C. `__str__()`
 - D. `__call__()`
- Answer: A**
-

7. What is method overloading in Python?

- A. Defining multiple methods with the same name but different signatures
 - B. Overriding a method from a parent class
 - C. Defining a method with no arguments
 - D. Python does not support method overloading
- Answer: D**
-

8. What will the following code output?

```
class Parent:
    def show(self):
        print("Parent class")
```



```
class Child(Parent):  
    pass  
  
c = Child()  
c.show()
```

- A. Child class
- B. Parent class
- C. None
- D. Error

Answer: B

9. What is the result of the following code?

```
class Vehicle:  
    def __init__(self, make, model):  
        self.make = make  
        self.model = model  
  
    def display(self):  
        return f'{self.make} {self.model}'  
  
v = Vehicle("Toyota", "Corolla")  
print(v.display())
```

- A. Toyota Corolla
- B. Toyota
- C. Corolla
- D. None

Answer: A

10. What is the purpose of the `__str__()` method in Python?

- A. To define the string representation of an object
- B. To initialize the object
- C. To compare objects
- D. To define the behavior of an object

Answer: A

11. What is the output of the following code?

```
class A:  
    def __init__(self, x):  
        self.x = x  
  
    def display(self):  
        return self.x
```

```
a = A(10)
print(a.display())
```

- A. 10
- B. None
- C. Error
- D. a.display()

Answer: A

12. Which concept allows the creation of new classes based on existing classes?

- A. Encapsulation
- B. Inheritance
- C. Polymorphism
- D. Abstraction

Answer: B

13. What will the following code output?

```
class MyClass:
    def __init__(self, name):
        self.name = name

    def __del__(self):
        print(f'{self.name} is being deleted')

obj = MyClass("TestObject")
del obj
```

- A. TestObject is being deleted
- B. None
- C. Error
- D. TestObject

Answer: A

14. Which of the following is true about Python classes?

- A. Class methods can be called without creating an object
- B. Python does not support inheritance
- C. A class can inherit from only one parent class
- D. Classes are not used to define object templates

Answer: A

15. What will the following code output?

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

d = Dog()
d.speak()
```

- A. Animal speaks
- B. Dog barks
- C. None
- D. Error

Answer: B

16. What is the purpose of the `super()` function in Python?

- A. It allows calling methods from the parent class
- B. It creates a new instance of the parent class
- C. It overrides a method in the current class
- D. It initializes the parent class

Answer: A

17. What is the output of this code?

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def __str__(self):
        return f'{self.brand} {self.model}'

car = Car("Honda", "Civic")
print(car)
```

- A. Honda Civic
- B. None
- C. <__main__.Car object at 0x123>
- D. Car

Answer: A

18. What does encapsulation refer to in Object-Oriented Programming?

- A. Hiding the internal state of an object
- B. Creating an instance of a class
- C. Allowing inheritance of methods
- D. Implementing an interface

Answer: A

19. What will the following code output?

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __repr__(self):
        return f"Employee({self.name}, {self.salary})"

e = Employee("John", 50000)
print(e)
```

- A. Employee(John, 50000)
- B. John 50000
- C. None
- D. Error

Answer: A

20. Which of the following is true about polymorphism in Python?

- A. It allows a function to accept arguments of different types
- B. It allows a class to have multiple constructors
- C. It enables a class to inherit methods from multiple classes
- D. It requires objects of the same type

Answer: A

21. Which of the following is true for Python inheritance?

- A. Python does not support multiple inheritance
- B. A child class can inherit from multiple parent classes
- C. Python supports only single inheritance
- D. The child class cannot access parent class methods

Answer: B

22. What will be the result of this code?

```
class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

circle = Circle(5)
print(circle.area())
```

- A. $3.14 * 5 * 5$
- B. None
- C. 25
- D. 78.5

Answer: D

23. Which of the following is the correct way to define an abstract method in Python?

- A. `def method(self): pass`
- B. `@abstractmethod def method(self): pass`
- C. `def method(self) {}`
- D. `abstract method(self): pass`

Answer: B

24. Which of the following can be a class attribute in Python?

- A. Instance variables
- B. Methods
- C. Constants
- D. Both B and C

Answer: D

25. What is the output of the following code?

```
class Base:

def show(self): print("Base class")

class Derived(Base): def show(self): print("Derived class")
```

```
obj = Derived() obj.show()
```

```
**A.** `Base class`  
**B.** `Derived class`  
**C.** `None`  
**D.** `Error`  
**Answer:** B
```

Here are 25 hard MCQs covering **Session 13 & 14: Advanced Topics in Python Programming**, with answers:

1. What will the following code output?

```
x = [1, 2, 3]  
y = [4, 5, 6]  
z = zip(x, y)  
print(list(z))
```

- A. [(1, 4), (2, 5), (3, 6)]
- B. [(1, 2), (3, 4), (5, 6)]
- C. [1, 2, 3]
- D. Error

Answer: A

2. Which of the following is the correct way to use list comprehensions with conditions?

- A. [x for x in range(10) if x % 2 == 0]
- B. [x if x % 2 == 0 for x in range(10)]
- C. [x in range(10) if x % 2 == 0]
- D. [for x in range(10) if x % 2 == 0]

Answer: A

3. What will the following code output?

```
from functools import reduce  
result = reduce(lambda x, y: x + y, [1, 2, 3, 4])  
print(result)
```

- A. 10
- B. 24
- C. None

D. Error

Answer: A

4. What is the output of the following code?

```
a = [1, 2, 3, 4]
b = iter(a)
print(next(b))
```

A. 1

B. None

C. [1]

D. Error

Answer: A

5. What does the `@staticmethod` decorator do in Python?

A. It defines a method that does not take `self` or `cls` as the first parameter

B. It defines a method that takes the class as a parameter

C. It is used for defining static class variables

D. It is used to instantiate the class

Answer: A

6. What is the result of the following code?

```
def my_function(a: int, b: float) -> float:
    return a + b

print(my_function(2, 3.5))
```

A. 5.5

B. None

C. 2.0

D. Error

Answer: A

7. What is the output of the following code?

```
class MyClass:
    def __init__(self, x):
        self.x = x

    def __call__(self, value):
```

```
        return self.x + value

obj = MyClass(5)
print(obj(10))
```

- A. None
- B. 15
- C. 5
- D. Error

Answer: B

8. What does the `yield` keyword do in Python?

- A. It creates a generator function
- B. It creates a regular function
- C. It returns a function object
- D. It raises an exception

Answer: A

9. Which of the following is true about Python generators?

- A. Generators are iterators
- B. Generators are memory efficient
- C. Generators are functions that return an iterable sequence of items
- D. All of the above

Answer: D

10. What is the output of the following code?

```
def foo():
    try:
        return 1
    finally:
        return 2

print(foo())
```

- A. 1
- B. None
- C. 2
- D. Error

Answer: C

11. What will the following code output?

```
import sys
print(sys.getrecursionlimit())
```

- A. 1000
- B. 3000
- C. 100
- D. Error

Answer: A

12. What is the purpose of the `@property` decorator in Python?

- A. It allows a method to be accessed like an attribute
- B. It is used to define class variables
- C. It marks a class method as static
- D. It modifies a function's behavior

Answer: A

13. What will be the output of the following code?

```
def test(a, b, c=5):
    print(a, b, c)

test(1, 2)
```

- A. 1 2 5
- B. 1 2
- C. None
- D. Error

Answer: A

14. Which of the following is used for type hinting in Python?

- A. `@type`
- B. `->`
- C. `#type`
- D. `<<type>>`

Answer: B

15. Which function is used to generate the hash value of an object in Python?

- A. `hash()`
- B. `id()`
- C. `gethash()`
- D. `hashcode()`

Answer: A

16. What is the result of the following code?

```
class MyClass:
    def __init__(self):
        self.x = 10

    def __repr__(self):
        return f'MyClass({self.x})'

obj = MyClass()
print(obj)
```

- A. `MyClass(10)`
- B. `10`
- C. `None`
- D. `Error`

Answer: A

17. Which of the following is true about the `__del__` method in Python?

- A. It is called when an object is garbage collected
- B. It is used to define an object's string representation
- C. It is used to initialize an object
- D. It is not supported in Python

Answer: A

18. What is the purpose of the `__slots__` mechanism in Python?

- A. To limit the attributes of an object
- B. To define instance methods
- C. To enable multiple inheritance
- D. To define static methods

Answer: A

19. What is the result of the following code?

```
x = {'name': 'John', 'age': 25}
```

```
y = x
y['age'] = 26
print(x)
```

- A. {'name': 'John', 'age': 26}
- B. {'name': 'John', 'age': 25}
- C. None
- D. Error

Answer: A

20. What does the `itertools.chain()` function do?

- A. It joins multiple iterables into one iterable
- B. It creates an infinite sequence
- C. It filters out specific elements from an iterable
- D. It maps elements in an iterable

Answer: A

21. Which of the following is a valid method for creating a class in Python?

- A. `class ClassName{}`
- B. `def ClassName():`
- C. `class ClassName:`
- D. `object ClassName:`

Answer: C

22. Which Python function allows you to convert an object into its string representation?

- A. `str()`
- B. `repr()`
- C. `print()`
- D. `str_repr()`

Answer: B

23. Which of the following is a feature of a Python lambda function?

- A. It can have multiple expressions
- B. It is always named
- C. It is a one-line function

D. It cannot accept arguments

Answer: C

24. What will the following code output?

```
def my_gen():  
    yield 1  
    yield 2  
    yield 3  
  
gen = my_gen()  
print(next(gen))
```

A. 1

B. None

C. Error

D. my_gen()

Answer: A

25. Which of the following will allow creating a thread in Python?

A. threading.Thread(target=function)

B. Thread.create(target=function)

C. Thread.start(target=function)

D. thread(target=function)

Answer: A

Session 15: Libraries and Functional Programming, with answers:

1. What is the purpose of the `functools` module in Python?

A. To define functions that can be used in functional programming

B. To implement object-oriented programming features

C. To support regular expression matching

D. To create graphical user interfaces

Answer: A

2. What will the following code output?

```
import math  
print(math.factorial(5))
```

- A. 120
- B. 5
- C. 25
- D. None

Answer: A

3. Which of the following is true about the `filter()` function in Python?

- A. It filters elements from a list based on a condition
- B. It transforms elements in an iterable
- C. It combines two lists into one
- D. It iterates over a range of numbers

Answer: A

4. Which function from the `itertools` module can be used to create an infinite sequence?

- A. `cycle()`
- B. `repeat()`
- C. `count()`
- D. All of the above

Answer: D

5. What is the result of the following code?

```
from functools import reduce
result = reduce(lambda x, y: x * y, [1, 2, 3, 4])
print(result)
```

- A. 10
- B. 24
- C. None
- D. Error

Answer: B

6. Which of the following functional programming concepts does Python support?

- A. Higher-order functions
- B. Closures
- C. Lambda functions

D. All of the above

Answer: D

7. What will the following code output?

```
from functools import partial
def power(base, exponent):
    return base ** exponent

cube = partial(power, exponent=3)
print(cube(2))
```

A. 8

B. 6

C. 3

D. Error

Answer: A

8. Which of the following is used to map a function to all items in an iterable?

A. map()

B. filter()

C. reduce()

D. zip()

Answer: A

9. What is the purpose of lambda functions in Python?

A. To define anonymous functions

B. To define recursive functions

C. To define regular functions with a name

D. To define functions that always return a constant value

Answer: A

10. What is the output of the following code?

```
def double(x):
    return x * 2

nums = [1, 2, 3, 4]
result = list(map(double, nums))
print(result)
```

- A. [1, 2, 3, 4]
- B. [2, 4, 6, 8]
- C. [2, 3, 4, 5]
- D. None

Answer: B

11. Which of the following modules is used for regular expressions in Python?

- A. regex
- B. re
- C. exp
- D. regexp

Answer: B

12. What is the result of the following code?

```
import operator
nums = [2, 4, 6]
result = reduce(operator.add, nums)
print(result)
```

- A. 12
- B. 14
- C. 24
- D. None

Answer: A

13. What is a key feature of functional programming in Python?

- A. Functions are first-class citizens
- B. Classes are used to encapsulate functions
- C. All functions must be named
- D. Functions can only return integers

Answer: A

14. What will the following code output?

```
from itertools import permutations
p = permutations([1, 2, 3])
print(list(p))
```

- A. [(1, 2, 3), (2, 1, 3), (3, 2, 1)]
- B. [(1, 2), (2, 3), (1, 3)]
- C. [(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)]
- D. None

Answer: C

15. Which of the following is used to create a list of tuples with all possible combinations from multiple iterables?

- A. `itertools.product()`
- B. `itertools.combinations()`
- C. `itertools.chain()`
- D. `itertools.zip_longest()`

Answer: A

16. What will the following code output?

```
from functools import reduce
result = reduce(lambda x, y: x - y, [10, 5, 2])
print(result)
```

- A. 3
- B. 5
- C. 7
- D. Error

Answer: C

17. Which of the following is a feature of Python's `filter()` function?

- A. It returns an iterable that includes elements based on a condition
- B. It returns a new list with all elements
- C. It iterates over a fixed set of numbers
- D. It returns a single result from a list

Answer: A

18. What will the following code output?

```
from operator import mul
result = reduce(mul, [1, 2, 3, 4])
print(result)
```


- A. 10
- B. 24
- C. None
- D. 1

Answer: B

19. Which of the following is used to combine multiple iterables into a single iterable in Python?

- A. `itertools.zip()`
- B. `itertools.chain()`
- C. `itertools.product()`
- D. `itertools.combine()`

Answer: B

20. What will the following code output?

```
def square(x):  
    return x ** 2  
  
result = list(map(square, [1, 2, 3, 4]))  
print(result)
```

- A. `[1, 4, 9, 16]`
- B. `[1, 2, 3, 4]`
- C. `[1, 4, 9]`
- D. None

Answer: A

21. Which of the following libraries is commonly used for numerical computations and working with large datasets in Python?

- A. `numpy`
- B. `math`
- C. `functools`
- D. `collections`

Answer: A

22. Which Python module would you use to work with JSON data?

- A. `json`
- B. `pickle`

C. `yaml`

D. `re`

Answer: A

23. Which of the following functional programming concepts can be implemented using `reduce()` ?

A. Accumulating a result over an iterable

B. Applying a condition to filter elements

C. Mapping a function over an iterable

D. Combining two iterables

Answer: A

24. What is the function of the `itertools.islice()` method?

A. It returns an iterator that generates a sliced portion of an iterable

B. It creates an infinite iterator

C. It combines elements from multiple iterables

D. It creates combinations of elements from an iterable

Answer: A

25. Which of the following is true about Python's `reduce()` function?

A. It applies a binary function cumulatively to the items in an iterable

B. It is used to create an iterable from a sequence of numbers

C. It works only on strings

D. It maps a function over an iterable

Answer: A

Session 16: Exploit Development Techniques and Writing Plugins in Python, with answers:

1. What is the main purpose of an exploit in computer security?

A. To find vulnerabilities and patch them

B. To take advantage of a vulnerability to gain unauthorized access

C. To detect malware

D. To encrypt data securely

Answer: B

2. What is a buffer overflow vulnerability?

- A. A condition where a program runs out of memory
- B. A vulnerability where a program writes more data to a buffer than it can handle
- C. A vulnerability related to invalid user input
- D. A condition that causes a program to freeze

Answer: B

3. In Python, which module is used for network communication in exploit development?

- A. `socket`
- B. `http.server`
- C. `subprocess`
- D. `os`

Answer: A

4. Which Python method can be used to create a malicious payload in an exploit?

- A. `os.system()`
- B. `socket.socket()`
- C. `subprocess.run()`
- D. `exec()`

Answer: D

5. What is the purpose of the `ptrace` system call in exploit development?

- A. To create a secure environment for running code
- B. To trace the execution of a program for debugging or manipulation
- C. To perform memory encryption
- D. To interact with network protocols

Answer: B

6. Which of the following best describes a Reverse Shell?

- A. A program that allows an attacker to remotely execute commands on a victim machine
- B. A program that prevents unauthorized access to a system
- C. A method to prevent buffer overflow attacks

D. A process that logs all network activity

Answer: A

7. Which module is commonly used in Python for working with file operations in exploit development?

A. `fileinput`

B. `os`

****C. `shutil`**

D. `tempfile`

Answer: B

8. What is a "plugin" in the context of Python programming for exploit development?

A. A third-party library that is used for data encryption

B. A modular piece of code that extends the functionality of a larger application

C. A function that runs in a separate thread

D. A tool used to identify vulnerabilities in a system

Answer: B

9. What is the most common method for injecting a payload in a buffer overflow attack?

A. Command-line arguments

B. Network packets

C. Stack smashing with excess data in the input buffer

D. Memory manipulation using the `os` module

Answer: C

10. What is a "NOP sled" in exploit development?

A. A sequence of NOP (No Operation) instructions to facilitate the execution of a payload

B. A method of obfuscating code

C. A tool for generating secure passwords

D. A technique for hiding malicious payloads

Answer: A

11. What is an example of a common Python tool for exploit development and testing?

- A. Scapy
- B. Flask
- C. Django
- D. Tkinter

Answer: A

12. How does a Python-based exploit typically achieve privilege escalation?

- A. By exploiting improper file permissions
- B. By using the `subprocess` module to execute malicious system commands
- C. By leveraging buffer overflow vulnerabilities
- D. All of the above

Answer: D

13. Which function is often used to execute system-level commands in Python during an exploit?

- A. `subprocess.run()`
- B. `os.system()`
- C. `exec()`
- D. `os.popen()`

Answer: B

14. In exploit development, what does the `socket.connect()` method do?

- A. It establishes a connection to a specified IP address and port number
- B. It terminates a connection
- C. It listens for incoming connections
- D. It sends an encrypted message over the network

Answer: A

15. Which of the following is an example of a remote code execution (RCE) vulnerability?

- A. Command injection
- B. Buffer overflow
- C. Format string vulnerabilities

D. All of the above

Answer: D

16. What is a "web shell"?

A. A shell that runs on the web server allowing remote command execution

B. A browser-based tool to check for exploits

C. A security tool to prevent remote attacks

D. A method for encrypting web traffic

Answer: A

17. Which Python method is used to execute commands remotely on a machine in exploit development?

A. `subprocess.Popen()`

B. `socket.recv()`

C. `os.system()`

D. `os.fork()`

Answer: A

18. In writing Python plugins for security tools, which design principle is most commonly followed?

A. Modularity

B. Obfuscation

C. Encryption

D. Reusability

Answer: A

19. What is the primary function of the `os.fork()` method in exploit development?

A. To execute a system command

B. To create a child process for executing a malicious payload

C. To manipulate system files

D. To execute a Python script remotely

Answer: B

20. What is the purpose of the `ctypes` module in Python when writing exploits?

- A. To interact with system-level APIs and data structures
- B. To manage large datasets
- C. To encode and decode messages securely
- D. To optimize memory usage in Python applications

Answer: A

21. What is the result of the following Python code when writing an exploit?

```
import socket
s = socket.socket()
s.connect(('localhost', 8080))
s.send(b'GET / HTTP/1.1\r\n')
```

- A. It sends a malicious HTTP request to a server
- B. It opens a connection to a specified IP and port
- C. It returns the current date and time
- D. It crashes the Python interpreter

Answer: B

22. Which of the following Python modules is useful for fuzz testing in exploit development?

- A. fuzzing
- B. unittest
- C. pytest
- D. hypothesis

Answer: D

23. What is the purpose of "shellcode" in exploit development?

- A. It is used to encode system commands
- B. It is a sequence of instructions used to execute malicious actions on a target system
- C. It is used to filter incoming data packets
- D. It prevents buffer overflow vulnerabilities

Answer: B

24. What does a "format string vulnerability" allow an attacker to do?

- A. Inject shellcode into a running program
- B. Overwrite memory with custom values
- C. Read or write arbitrary memory locations

D. All of the above

Answer: D

25. What will the following Python code achieve in an exploit development context?

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(('127.0.0.1', 1234))
s.send(b'GET / HTTP/1.1\r\nHost: localhost\r\n\r\n')
```

A. It attempts to establish a connection to a server and send an HTTP request

B. It creates a local web server

C. It reads data from the network

D. It encrypts the message before sending

Answer: A

Session 17: Exploit Analysis Automation and Debugging Basics, with answers:

1. What is the primary purpose of exploit analysis?

A. To patch vulnerabilities

B. To detect and understand security weaknesses in a system

C. To monitor network traffic

D. To encrypt sensitive data

Answer: B

2. Which Python module can be used to automate network traffic analysis during exploit development?

A. os

B. scapy

C. re

D. math

Answer: B

3. In debugging, what does the term "breakpoint" refer to?

A. A point in code where execution is paused to inspect variables and state

B. A point where a program crashes unexpectedly

- C. A point where the program generates output
- D. A place where the program encrypts data

Answer: A

4. Which of the following tools is primarily used for automated exploit analysis and debugging?

- A. gdb
- B. vim
- C. ipython
- D. Flask

Answer: A

5. What is the role of fuzz testing in exploit analysis automation?

- A. To encrypt messages
- B. To find bugs by sending random or malformed input to a program
- C. To secure network communications
- D. To fix memory leaks

Answer: B

6. Which Python library can be used to automate repetitive tasks and integrate external tools for exploit analysis?

- A. os
- B. subprocess
- C. re
- D. threading

Answer: B

7. In exploit debugging, which of the following is typically used to trace a program's execution flow?

- A. Memory dumps
- B. Breakpoints
- C. Print statements
- D. All of the above

Answer: D

8. What does the term "segfault" (segmentation fault) indicate in exploit analysis?

- A. A program crashed due to illegal memory access
- B. The program successfully executed a malicious payload
- C. An infinite loop occurred in the program
- D. The program returned an incorrect output

Answer: A

9. Which Python function is used to launch an external command and get its output, which can help automate exploit analysis?

- A. `os.popen()`
- B. `subprocess.call()`
- C. `subprocess.run()`
- D. `sys.exit()`

Answer: C

10. What is the purpose of stack smashing in exploit development?

- A. To overload a program with too many function calls
- B. To overwrite the return address in the stack and execute arbitrary code
- C. To execute system commands
- D. To test memory leak vulnerabilities

Answer: B

11. Which of the following techniques is typically used for debugging a Python program?

- A. `gdb`
- B. `print` statements
- C. `sys.settrace()`
- D. `logging` module

Answer: B

12. In exploit analysis, what is a "payload"?

- A. The part of an exploit that carries out the intended attack
- B. A tool for fixing bugs in code
- C. A network communication protocol

D. A script to detect malware

Answer: A

13. What is the first step in automated exploit analysis?

A. Finding a vulnerability to exploit

B. Setting breakpoints to halt execution

C. Setting up a testing environment with automated scripts

D. Writing the payload

Answer: C

14. Which Python module can help automate the extraction of memory dumps in exploit analysis?

A. `psutil`

B. `subprocess`

C. `pywin32`

D. `ctypes`

Answer: A

15. What is the role of `gdb` in debugging?

A. It automates network traffic analysis

B. It is used to test Python scripts for errors

C. It is a debugger for C/C++ programs to inspect memory and control execution

D. It provides GUI-based debugging tools for Python

Answer: C

16. What is the significance of a core dump in debugging?

A. It contains information about the memory state of a crashed program, useful for post-mortem debugging

B. It prevents system crashes during exploits

C. It is a method for obfuscating code

D. It encrypts the program's execution

Answer: A

17. Which Python function can help automate debugging by showing detailed information about a function's execution?

- A. `pdb.set_trace()`
- B. `traceback.print_exc()`
- C. `logging.debug()`
- D. `print()`

Answer: A

18. What is "race condition" in the context of exploit analysis?

- A. A vulnerability where multiple processes access shared resources at the same time, leading to unexpected results
- B. A type of memory corruption vulnerability
- C. A system crash due to improper code execution
- D. A failure to write to a file

Answer: A

19. Which Python library is helpful for interacting with system-level processes during automated exploit analysis?

- A. `sys`
- B. `subprocess`
- C. `io`
- D. `ctypes`

Answer: B

20. What is the primary function of `pdb` in Python?

- A. To perform network traffic analysis
- B. To provide a debugger for inspecting and controlling the flow of a Python program
- C. To generate cryptographic keys
- D. To automate memory dump extraction

Answer: B

21. In the context of exploit analysis, what is a "shellcode"?

- A. The part of the exploit that connects back to the attacker's machine
- B. A code used to obfuscate the exploit's real purpose
- C. A piece of code that is executed when a vulnerability is triggered
- D. A script that scans a system for vulnerabilities

Answer: C

22. Which of the following debugging methods can help trace function calls and arguments in Python?

- A. `pdb.set_trace()`
- B. `logging.debug()`
- C. `sys.settrace()`
- D. all of the above

Answer: D

23. What is the purpose of fuzz testing in exploit analysis?

- A. To optimize the performance of code
- B. To automatically find security vulnerabilities by inputting random or malformed data
- C. To encrypt data
- D. To fix memory leaks

Answer: B

24. What would you use the `logging` module for in exploit analysis?

- A. To automate the execution of exploits
- B. To monitor and record events during the execution of an exploit or debug session
- C. To prevent buffer overflow vulnerabilities
- D. To automate the reverse engineering process

Answer: B

25. What is a common cause of segmentation faults during exploit analysis?

- A. Buffer overflow
- B. Invalid system call
- C. Invalid network configuration
- D. Infinite loop

Answer: A
